

REPORT 04

<최종>



충남대학교

이름/학번 : 오석준/202002810

날짜 : 2024. 12. 09

이메일 : lose8468@naver.com

핸드폰번호 : 01073028468

본 프로젝트는 객체지향프로그래밍 과목에서 Java Swing을 활용해 제작한 시뮬레이션 게임입니다. 게임 디자인 관점에서 캐릭터 상태값, 목표 조건, 랜덤 이벤트, 보상 구조를 직접 설계하고 구현한 경험을 정리했습니다.

※ - 내 프로젝트의 개발 난이도(얼마나 어려운 일을 하였는가) : 3(중간) -이번 프로젝트는 게임 시뮬레이션의 기본 구조를 구현하는 데 초점을 맞췄습니다. 게임의 전체 흐름을 구현하는 데 있어 복잡한 알고리즘이나 고급 기술이 요구되지는 않았기 때문에 개발 난이도를 "중간"으로 평가

- 내 프로젝트의 완성도(하려던 계획을 얼마나 완성하였는가) : 3(중간) -몇 가지 추가적인 개선 가능성이 남아 있어 완성도를 "중간"으로 평가하였습니다. ex) 만족도가 높은 병사에게 추가 보상(휴가 지급) , 이벤트와 훈련 시스템의 다양화.

- 내 프로젝트의 사용자 고려한 친절성(프로그램 시작시키고, 얼마나 직관적으로 사용할 수 있게 만들었는가) : 4(쉽게 알수 있음)- 병사들의 상태(만족도, 훈련 점수, 휴가 일수)를 명확히 표시하여 현재 상황을 쉽게

파악할 수 있도록 했습니다. 랜덤 이벤트가 발생했을 때 사용자에게 팝업 메시지가 나오게 하였습니다. 병사별 상태를 관리할 수 있는 버튼과 매일 상태를 업데이트하는 버튼을 분리하였습니다.

- 책 이외의 다른 소스코드에 대한 의존도 : 3(중간)세부적인 부분은 참고 코드를 변형하여 사용하였습니다. 특정 코드에 전적으로 의존하지 않았고, 참고한 자료를 적절히 응용하여 구현했기 때문에 "중간" 수준으로 평가하였습니다.

- 최종적으로 내가 갖다 쓴 source 들 목록 및 출처 (그림, 음악, 코드, 내용 등등)

/*김태우.png 출처:

https://imgnews.pstatic.net/image/108/2008/12/09/2008120914523202415_1.jpg?type=w647

최종훈.png 출처 :

https://postfiles.pstatic.net/MjAxNzEyMDFfMTE1/MDAxNTEyMTE0NjgzNDQ2.rZPwXK59F8KZtjFLkz6zWojPZg5RZBgQBKOSkbO7cgg.DDtQG_r67uuMD22yyd_nvrASMD1B9Tr-EXl18KhsHvog.JPEG.sjw608/%EB%A7%90%EB%85%84%EB%B3%91%EC%9E%A5.jpg?type=w966

굳건이.png 출처 :

https://upload.wikimedia.org/wikipedia/commons/thumb/e/eb/Military_Manpower_Administration_Gutgeonil.png/200px-Military_Manpower_Administration_Gutgeonil.png

나백중.png 출처 :

<https://img.insight.co.kr/static/2019/02/13/700/97300lv5mvgz35ydca9u.jpg>

군.png 출처 :

<https://i.namu.wiki/i/8ZvD7obEx8NqEZMfmJPFTqinm>

<CY2-c-v7P409tLGvPbG0HSD9rx-AAEf0eBxB8vohWyHwR4nU5GrqaKjNZnsPQ.webp>

흑한기.png 출처:

https://dispatch.cdnser.be/wp-content/uploads/2017/02/20170207162821_1-2.jpg

군가2 출처 :

https://youtu.be/Nb8p3dfh0XA?si=KLaRQndz6qhTPGRn*/

Set 참고 출처 :

<https://blog.naver.com/heartflow89/220994601249>

JOptionPane 참고 출처: 책 p789 ~791

StringBuilder 참고 출처 :

<https://onlyfor-me-blog.tistory.com/317>

getscaledInstance 참조 출처:

<https://blog.naver.com/wonsukdream/30138796350>

setenabled 참조 출처:

<https://yoo11052.tistory.com/17>

1. 문제 정의

1) 시나리오 배경

전역을 50일 앞둔 4명의 병장들이 전역 20일 전 예정된 흑한기 훈련을 피하고자 한다.

병장들은 각자 휴가 10일을 보유하고 있으며, 이를 20일로 모아 흑한기 훈련을 회피하고 휴가를 떠나고 싶어 한다.

2) 문제 상황

병사들의 만족도와 훈련 점수를 관리해야 한다.

훈련 점수를 모으지 못하고 휴가를 20일을 다 모으지 못하면 흑한기 훈련을 강제로 받게 된다.

특정 조건을 충족했을 때 병사 상태를 업데이트하고 이를 사용자에게 명확히 보여줄 수 있어야 한다.

3) 요구 사항

병사들의 훈련 점수 관리: 훈련 점수 10점을 얻으면 휴가 1일 추가한다.

병사들의 만족도 관리: 훈련과 이벤트에 따라 만족도 변화한다.

휴가 20일을 달성한 병사는 "휴가 중" 상태로 전환한다.

랜덤 이벤트 발생으로 게임 진행에 변수 추가한다.

2. 해결 방법

1) 상태 초기화

관련 메소드: main, createStartScreen

병사 이름, 만족도, 휴가 일수, 훈련 점수 등을 초기화한다.

데이터를 HashMap으로 저장하여 각 병사의 상태를 효율적으로 관리한다.

첫 화면을 보여준다.

2) 게임 시작 및 시나리오 설명

관련 메소드: showIntroPopup, createShowGUI

게임 시작 시 병사들의 상황(혹한기 훈련 일정과 목표)을 사용자에게 설명한다.

사용자가 게임 진행을 선택하면 메인 메뉴로 이동하여 다양한 기능(스케줄 관리, 휴가 관리 등)을 사용할 수 있다.

3) 병사 상태 업데이트

관련 메소드: updateSoldierStates

매일 병사의 상태를 업데이트하며, 다음과 같은 상황을 수행한다:

하루가 지날때마다 병사들의 만족도를 매일 1씩 감소시킨다.

이 메소드는 nextDayButton을 통해 호출됩니다.

훈련 점수가 10점 이상일 경우: 훈련 점수를 차감하고 휴가를 1일 추가한다. 만족도를 조금 회복시킨다.

휴가 20일을 달성한 경우: 병사를 "휴가 중" 상태로 전환하고 해당 병사의 버튼을 비활성화한다.

4) 스케줄 관리

관련 메소드: `manageSchedule`

병사들의 현재 상태를 확인하여 사용자에게 보여준다.

병사가 "휴가 중" 상태인지, "훈련 대기 중" 상태인지 확인하고 메시지를 업데이트한다.

휴가를 20일 달성한 병사는 스케줄 관리 화면에서 자동으로 "휴가 중"으로 표시된다.

5)만족도 확인

관련 메소드: `showSatisfactionStatus`

모든 병사의 만족도를 사용자에게 보여준다.

6) 휴가 관리

관련 메소드: `manageLeave`

병사들의 현재 휴가 일수를 사용자에게 보여준다.

휴가가 20일에 도달하지 못한 병사는 훈련에 참여해야 한다.

7) 랜덤 이벤트 발생

관련 메소드: `triggerRandomEvent`

랜덤 이벤트를 통해 병사 상태에 변화를 준다:

야간 경계: 만족도 -5, 훈련 점수 +3

체력 단련: 만족도 +5

전술 훈련: 만족도 +3, 훈련 점수 +5

단발령: 만족도 -15

랜덤 이벤트는 7일마다 자동 발생하며, 사용자에게 이벤트 내용을 알린다.

이벤트 발생으로 병사의 만족도와 훈련 점수가 변경된다.

8) 게임 종료 및 결과 확인

관련 메소드: showGameResults

30일이 지나면 게임이 종료되고, 병사들의 최종 상태를 사용자에게 보여준다:

휴가 20일을 달성한 병사는 "휴가자"로 표시하였습니다.

그렇지 못한 병사는 흑한기 참가로 표시하였습니다.

병사별 만족도, 훈련 점수, 휴가 일수를 모두 종합하여 사용자에게 결과를 출력한다.

9) 음악 및 효과 관리

관련 메소드: playMusic, stopMusic

배경 음악을 재생하고 끄는 기능 제공.

게임의 몰입도를 높이기 위해 군가를 배경 음악으로 설정하였다

3. 최종 소스 코드

/*김태우.png 출처:

https://imgnews.pstatic.net/image/108/2008/12/09/2008120914523202415_1.jpg?type=w647

최종훈.png 출처 :

https://postfiles.pstatic.net/MjAxNzEyMDFfMTE1/MDAxNTEyMTE0NjgzNDQ2.rZPwXK59F8KZtjFLkz6zWojPZg5RZBgQBEEKOSkbO7cgg.DDtQG_r67uuMD22yyd_nvrASMD1B9Tr-EXl18KhsHvog.JPEG.sjw608/%EB%A7%90%EB%85%84%EB%B3%91%EC%9E%A5.jpg?type=w966

굳건이.png 출처 :

https://upload.wikimedia.org/wikipedia/commons/thumb/e/eb/Military_Manpower_Administration_Gutgeonil.png/200px-Military_Manpower_Administration_Gutgeonil.png

나백중.png 출처 :

<https://img.insight.co.kr/static/2019/02/13/700/97300lv5mvgz35ydca9u.jpg>

군.png 출처 :

<https://i.namu.wiki/i/8ZvD7obEx8NqEZMfmJPFTqinmCY2-c-v7P409tLGvPbG0HSD9rx-AAEf0eBxB8vohWyHwR4nU5GrqaKjNZnsPQ.webp>

흑한기.png 출처:

https://dispatch.cdnser.be/wp-content/uploads/2017/02/20170207162821_1-2.jpg

군가2 출처 :

https://youtu.be/Nb8p3dfh0XA?si=KLaRQndz6qhTPGRn*/

/* 시나리오 제대로 잡기

제목- 병장 흑한기 탈출 시뮬레이션

이벤트 - 두발 단장(단발령) 추가

상황설명- 전역일이 50일 남은 말년 병장 4명이 있다.
전역일 20일 전에 흑한기 훈련이 예정. 이들은 병장이기에
절대 흑한기 훈련을 받지 않으려고 함.
병장 4명이 휴가 10일을 각각 동일하게 가지고 있다.
휴가 20일을 다모으면 흑한기 훈련을 가지 않는 시스템을
만들자.
휴가를 못모은 사람은 흑한기 훈련을 받다가 휴가를
나가는걸로 한다.*/*

//음악 파일 넣었습니다. 중단 버튼도 넣었습니다.
//synchronized는 활용을 잘 못하였습니다.
//휴가 20일 달성 시: 해당 병사를 onLeaveSoldiers에
추가해 휴가 상태로 전환. 버튼 비활성화로 추가 조작
불가
//훈련 점수 달성 시 만족도 회복 가능.
//훈련점수 10점 씩 휴가 1일
// 나머지 메소드들도 조금씩 수정

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.util.*;
import java.util.List;
import javax.sound.sampled.*;
import java.io.File;
import java.io.IOException;

public class MilitaryLifeSimulation_6 {

    private static Clip clip;

    static class StartButtonActionListener
```

```

implements ActionListener {
    private JFrame frame;

    public StartButtonActionListener(JFrame
frame) {
        this.frame = frame;
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        playMusic("src/music/군가2.wav");
        showIntroPopup(frame);
    }
}

    static class MuteButtonActionListener
implements ActionListener {
        @Override
        public void actionPerformed(ActionEvent e) {
            stopMusic();
        }
    }

    static class ScheduleActionListener implements
ActionListener {
        private JFrame frame;

        public ScheduleActionListener(JFrame frame)
{
            this.frame = frame;
        }

        @Override
        public void actionPerformed(ActionEvent e) {
            manageSchedule(frame);
        }
    }

    static class SatisfactionActionListener
implements ActionListener {
        private JFrame frame;

```

```

    public SatisfactionActionListener(JFrame
frame) {
        this.frame = frame;
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        showSatisfactionStatus(frame);
    }
}

```

```

    static class LeaveActionListener implements
ActionListener {
        private JFrame frame;

        public LeaveActionListener(JFrame frame) {
            this.frame = frame;
        }

        @Override
        public void actionPerformed(ActionEvent e) {
            manageLeave(frame);
        }
    }
}

```

```

    static class RandomEventActionListener
implements ActionListener {
        private JFrame frame;

        public RandomEventActionListener(JFrame
frame) {
            this.frame = frame;
        }

        @Override
        public void actionPerformed(ActionEvent e) {
            triggerRandomEvent(frame);
        }
    }
}

```

```

        static class ExitActionListener implements
        ActionListener {
            private JFrame frame;

            public ExitActionListener(JFrame frame) {
                this.frame = frame;
            }

            @Override
            public void actionPerformed(ActionEvent e) {
                exitApplication(frame);
            }
        }

```

```

        static class StartGameActionListener implements
        ActionListener {
            private JFrame frame;

            public StartGameActionListener(JFrame
            frame) {
                this.frame = frame;
            }

            @Override
            public void actionPerformed(ActionEvent e) {
                JOptionPane.showMessageDialog(frame,
                "게임을 시작합니다! 병사들을 관리하여 목표를
                달성하세요.", "게임 시작",
                JOptionPane.INFORMATION_MESSAGE);
                startGameLoop(frame);
            }
        }

```

```

        private static HashMap<String, Integer>
        satisfactionMap = new HashMap<>(); // 병사 만족도
        private static HashMap<String, Integer>
        leaveMap = new HashMap<>(); // 휴가 일수
        private static HashMap<String, Integer>
        taskPointsMap = new HashMap<>(); // 훈련 점수
        private static HashMap<String, String> rankMap
        = new HashMap<>(); // 계급 정보

```

```

        private static Set<String> onLeaveSoldiers = new
HashSet<>(); // 휴가 중인 병사 목록 Set 참고 출처 :
https://blog.naver.com/heartflow89/220994601249
        private static String[] soldierNames = { "최종훈",
"김태우", "굳건이", "나백중" };
        private static String[] soldierRanks = { "병장",
"병장", "병장", "병장" };
        private static String[] soldierImages = {
"src/images/최종훈.png", "src/images/김태우.png",
"src/images/굳건이.png", "src/images/나백중.png" };

        public static void main(String[] args) {
            // 초기 데이터 설정
            for (int i = 0; i < soldierNames.length; i++) {
                satisfactionMap.put(soldierNames[i], 100);
            }
            // 기본 만족도 100
            leaveMap.put(soldierNames[i], 10); //
            기본 휴가 10일
            taskPointsMap.put(soldierNames[i], 0); //
            기본 훈련 점수 0
            rankMap.put(soldierNames[i],
soldierRanks[i]);
        }

        // 시작 화면 실행
        createStartScreen();
    }

    private static void createStartScreen() {
        JFrame frame = new JFrame("병장 흑한기 탈출
시뮬레이션");

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLO
SE);

        frame.setSize(800, 600);

        try {
            ImageIcon backgroundImage = new
ImageIcon("src/images/군.png");
            JLabel backgroundLabel = new
JLabel(backgroundImage);

```

```

        backgroundLabel.setLayout(new
BorderLayout());
        frame.setContentPane(backgroundLabel);
    } catch (Exception ex) {
        System.out.println("배경 이미지를 로드할
수 없습니다: " + ex.getMessage());
    }

    JButton startButton = new JButton("시작하기");
    startButton.setFont(new Font("맑은 고딕",
Font.BOLD, 20));
    startButton.setBackground(Color.WHITE);
    startButton.setForeground(Color.BLACK);

    startButton.addActionListener(new
StartButtonActionListener(frame));

    JPanel buttonPanel = new JPanel();
    buttonPanel.setOpaque(false);
    buttonPanel.add(startButton);
    addMuteButton(buttonPanel);

    frame.add(buttonPanel,
BorderLayout.SOUTH);
    frame.setVisible(true);
}

private static void showIntroPopup(JFrame
frame) {
    JOptionPane.showMessageDialog(frame,
"전역일이 50일 남은 말년 병장 4명이 있습니다.\n전역일
20일 전에 혹한기 훈련이 예정되어있습니다.\n이들은 말년
병장이기에 '절대' 혹한기 훈련을 받지 않으려 하고
있습니다.\n", "상황 설명",
JOptionPane.INFORMATION_MESSAGE);

    int response =
JOptionPane.showConfirmDialog(frame, "다음으로
진행하시겠습니까?", "확인",
JOptionPane.YES_NO_OPTION); // JOptionPane 참고
출처: p789 ~791

```

```

        if (response == JOptionPane.YES_OPTION) {
            JOptionPane.showMessageDialog(frame,
"병장 4명이 휴가 10일을 각각 동일하게 가지고
있습니다.\n휴가 20일을 다 모으면 혹한기 훈련을 가지
않고 조기전역을 합니다.\n휴가를 못 모은 사람은 혹한기
훈련을 받다가 휴가를 나가는 안타까운
상황입니다.\n이들의 휴가와 훈련은 당신의 손에
맡겨져있습니다.", "상황 설명",
JOptionPane.INFORMATION_MESSAGE);
            createShowGUI(frame);
        }
    }

    private static void createShowGUI(JFrame frame)
    {
        frame.getContentPane().removeAll();
        frame.revalidate();
        frame.repaint();

        try {
            ImageIcon backgroundImage = new
ImageIcon("src/images/혹한기.png");
            JLabel backgroundLabel = new
JLabel(backgroundImage);
            backgroundLabel.setLayout(new
BorderLayout());
            frame.setContentPane(backgroundLabel);
        } catch (Exception ex) {
            System.out.println("메인 메뉴 배경
이미지를 로드할 수 없습니다: " + ex.getMessage());
        }

        JMenuBar menuBar = new JMenuBar();

        JMenu menu = new JMenu("메뉴");
        JMenuItem startGameMenuItem = new
JMenuItem("게임 시작");
        JMenuItem scheduleMenuItem = new
JMenuItem("병사 스케줄 관리");
        JMenuItem satisfactionMenuItem = new
JMenuItem("병사 만족도 확인");

```

```
        JMenuItem leaveMenuItem = new
JMenuItem("휴가 관리");
        JMenuItem randomEventMenuItem = new
JMenuItem("이벤트 발생");
        JMenuItem exitMenuItem = new
JMenuItem("종료");

        menu.add(startGameMenuItem);
        menu.add(scheduleMenuItem);
        menu.add(satisfactionMenuItem);
        menu.add(leaveMenuItem);
        menu.add(randomEventMenuItem);
        menu.addSeparator();
        menu.add(exitMenuItem);

        menuBar.add(menu);
        frame.setJMenuBar(menuBar);

        startGameMenuItem.addActionListener(new
StartGameActionListener(frame));
        scheduleMenuItem.addActionListener(new
ScheduleActionListener(frame));
        satisfactionMenuItem.addActionListener(new
SatisfactionActionListener(frame));
        leaveMenuItem.addActionListener(new
LeaveActionListener(frame));

        randomEventMenuItem.addActionListener(new
RandomEventActionListener(frame));
        exitMenuItem.addActionListener(new
ExitActionListener(frame));

        JPanel mutePanel = new JPanel();
        mutePanel.setOpaque(false);
        addMuteButton(mutePanel);
        frame.add(mutePanel, BorderLayout.SOUTH);

        frame.revalidate();
        frame.repaint();
```

```

        JOptionPane.showMessageDialog(frame, "메인
메뉴로 이동했습니다. 게임을 시작하거나 원하는 작업을
선택하세요.", "메인 메뉴",
JOptionPane.INFORMATION_MESSAGE);
    }

```

```

        private static void playMusic(String filePath) { //
참고 : p 809

```

```

            try {
                File audioFile = new File(filePath);
                AudioInputStream audioStream =
AudioSystem.getAudioInputStream(audioFile);
                clip = AudioSystem.getClip();
                clip.open(audioStream);
                clip.start();
            } catch (UnsupportedAudioFileException |
IOException | LineUnavailableException e) {
                System.out.println("음악을 재생할 수
없습니다: " + e.getMessage());
            }
        }

```

```

        private static void stopMusic() {
            if (clip != null && clip.isRunning()) {
                clip.stop();
            }
        }

```

```

        private static void addMuteButton(JPanel panel) {
            JButton muteButton = new JButton("음악
끄기");
            muteButton.setFont(new Font("맑은 고딕",
Font.BOLD, 20));
            muteButton.setBackground(Color.WHITE);
            muteButton.setForeground(Color.RED);
            muteButton.addActionListener(new
MuteButtonActionListener());
            panel.add(muteButton);
        }

```

```

        private static void manageSchedule(JFrame

```

```

frame) {
    StringBuilder scheduleMessage = new
    StringBuilder("병사 스케줄 관리:\n");
    for (String name : soldierNames) {
        if (onLeaveSoldiers.contains(name)) {

scheduleMessage.append(name).append(" - 휴가
중\n");
            } else if (leaveMap.get(name) >= 20) {

scheduleMessage.append(name).append(" - 휴가
중\n");
                onLeaveSoldiers.add(name); // 휴가
상태로 전환
            } else {

scheduleMessage.append(name).append(" - 훈련 대기
중\n");
                }
            }
        JOptionPane.showMessageDialog(frame,
scheduleMessage.toString(), "스케줄 관리",
JOptionPane.INFORMATION_MESSAGE);
    }

    private static void showSatisfactionStatus(JFrame
frame) {
        StringBuilder status = new
        StringBuilder("병사 만족도 현황:\n"); //StringBuilder
참고 출처 : https://onlyfor-me-blog.tistory.com/317
        for (String name : soldierNames) {
            status.append(name).append(" - 만족도:
").append(satisfactionMap.get(name)).append("\n");
        }
        JOptionPane.showMessageDialog(frame,
status.toString(), "만족도 확인",
JOptionPane.INFORMATION_MESSAGE);
    }

    private static void manageLeave(JFrame frame) {

```

```

        StringBuilder leaveStatus = new
StringBuilder("병사 휴가 현황:\n");
        for (String name : soldierNames) {
            leaveStatus.append(name).append(" -
남은 휴가: ").append(leaveMap.get(name)).append("\n");
        }
        JOptionPane.showMessageDialog(frame,
leaveStatus.toString(), "휴가 관리",
JOptionPane.INFORMATION_MESSAGE);
    }

```

```

        private static void triggerRandomEvent(JFrame
frame) {
            String[] events = {"야간 경계", "체력 단련",
"전술 훈련", "단발령"};
            Random random = new Random();
            String selectedEvent =
events[random.nextInt(events.length)];

```

```

            for (String name : soldierNames) {
                if (onLeaveSoldiers.contains(name))
continue;

```

```

                int satisfaction =
satisfactionMap.get(name);
                int points = taskPointsMap.get(name);
                switch (selectedEvent) {
                    case "야간 경계":
                        satisfaction -= 5;
                        points += 3;
                        break;
                    case "체력 단련":
                        satisfaction += 5;
                        break;
                    case "전술 훈련":
                        satisfaction += 3;
                        points += 5;
                        break;
                    case "단발령":
                        satisfaction -= 15;
                        break;

```

```

        }
        satisfactionMap.put(name,
Math.max(satisfaction, 0));
        taskPointsMap.put(name, points);
    }

    JOptionPane.showMessageDialog(frame, "랜덤
이벤트 발생: " + selectedEvent, "이벤트 발생",
JOptionPane.INFORMATION_MESSAGE);
}

    private static void exitApplication(JFrame frame)
{
    JOptionPane.showMessageDialog(frame,
"프로그램이 종료됩니다.", "종료",
JOptionPane.INFORMATION_MESSAGE);
    System.exit(0);
}

    private static synchronized void
updateSoldierStates(JPanel soldierPanel) {
    for (int i = 0; i < soldierNames.length; i++) {
        // 휴가 중인 병사는 최종 상태를 유지하며
업데이트
        if
(onLeaveSoldiers.contains(soldierNames[i])) {
            int leave =
leaveMap.get(soldierNames[i]);
            int satisfaction =
satisfactionMap.get(soldierNames[i]);
            int points =
taskPointsMap.get(soldierNames[i]);

            // soldierPanel에서 정보를 동기화
            JPanel individualPanel = (JPanel)
soldierPanel.getComponent(i);
            JLabel statsLabel = (JLabel)
individualPanel.getComponent(2);
            statsLabel.setText("만족도: " +

```

```

satisfaction + " | 훈련 점수: " + points + " | 휴가: " +
leave);
        continue;
    }

    int satisfaction =
satisfactionMap.get(soldierNames[i]);
    int points =
taskPointsMap.get(soldierNames[i]);
    int leave = leaveMap.get(soldierNames[i]);

    // 훈련 점수 및 만족도 업데이트
    if (points >= 10) {
        points -= 10;
        taskPointsMap.put(soldierNames[i],
points);

        leave += 1;
        leaveMap.put(soldierNames[i], leave);

        satisfaction += 5;
        satisfactionMap.put(soldierNames[i],
satisfaction);
    }

    // 휴가 20일 달성 시 처리
    if (leave >= 20) {
        JOptionPane.showMessageDialog(null,
soldierNames[i] + " 병사가 휴가 20일을 달성했습니다!
휴가를 떠납니다!", "휴가 이벤트",
JOptionPane.INFORMATION_MESSAGE);

        // 병사 비활성화
        JPanel individualPanel = (JPanel)
soldierPanel.getComponent(i);
        JButton actionButton = (JButton)
individualPanel.getComponent(3);
        actionButton.setEnabled(false); //
버튼 비활성화 참고 출처 :
https://yoo11052.tistory.com/17

```

```

        onLeaveSoldiers.add(soldierNames[i]);
// 휴가 중인 병사 목록에 추가
        continue;
    }

```

```

        satisfaction = Math.max(0, satisfaction -
1); // 매일 만족도 감소
        satisfactionMap.put(soldierNames[i],
satisfaction);

```

```

        // soldierPanel에서 업데이트된 정보 반영
        JPanel individualPanel = (JPanel)
soldierPanel.getComponent(i);
        JLabel statsLabel = (JLabel)
individualPanel.getComponent(2);
        statsLabel.setText("만족도: " + satisfaction
+ " | 훈련 점수: " + points + " | 휴가: " + leave);
    }
}

```

```

private static void showGameResults(JFrame
frame) {
    StringBuilder result = new StringBuilder("게임
종료! 최종 결과:\n");
    List<String> leaveAchievers = new
ArrayList<>();
    List<String> winterCampers = new
ArrayList<>();

```

```

    // 휴가 20일 모은 병사 처리
    for (String name : soldierNames) {
        int leaveDays = leaveMap.get(name);
        if (leaveDays >= 20) {
            leaveAchievers.add(name);
        } else {
            winterCampers.add(name);
        }
    }
}

```

```

// 비활성화된 병사 정보도 결과에 반영
for (String name : onLeaveSoldiers) {

```

```

        if (!leaveAchievers.contains(name)) {
            leaveAchievers.add(name);
        }
    }

    // 결과 출력
    for (String name : soldierNames) {
        int satisfaction =
satisfactionMap.get(name);
        int taskPoints =
taskPointsMap.get(name);
        int leaveDays = leaveMap.get(name);

        result.append(name).append(" - 만족도:
").append(satisfaction).append(", 훈련 점수:
").append(taskPoints).append(", 휴가:
").append(leaveDays).append("\n");
    }

    JOptionPane.showMessageDialog(frame,
result.toString(), "게임 결과",
JOptionPane.INFORMATION_MESSAGE);

    if (!leaveAchievers.isEmpty()) {
        for (String name : leaveAchievers) {

JOptionPane.showMessageDialog(frame, "휴가자 " +
name + " 그는 혹한기를 가지 않게되었습니다.
축하드립니다!", "결과",
JOptionPane.INFORMATION_MESSAGE);
        }
    }

    if (!winterCampers.isEmpty()) {
        for (String name : winterCampers) {

JOptionPane.showMessageDialog(frame, name + "
그는 혹한기 훈련을 받고 휴가를 나가게 되었습니다.
안타깝네요. ㅠㅠ", "결과",
JOptionPane.INFORMATION_MESSAGE);
        }
    }

```

```

    }

    createShowGUI(frame);
}

// showGameResults 호출 추가
private static void startGameLoop(JFrame frame)
{
    JPanel gamePanel = new JPanel(new
    BorderLayout());
    JLabel dayLabel = new JLabel("Day: 1",
    SwingConstants.CENTER);
    dayLabel.setFont(new Font("맑은 고딕",
    Font.BOLD, 24));
    gamePanel.add(dayLabel,
    BorderLayout.NORTH);

    JPanel soldierPanel = new JPanel(new
    GridLayout(2, 2, 10, 10));

    for (int i = 0; i < soldierNames.length; i++) {
        JPanel individualPanel = new JPanel(new
        BorderLayout());
        JLabel imageLabel = new JLabel();
        ImageIcon icon = new
        ImageIcon(soldierImages[i]);
        Image scaledImage =
        icon.getImage().getScaledInstance(150, 150,
        Image.SCALE_SMOOTH); //참조 출처:
https://blog.naver.com/wonsukdream/30138796350
        imageLabel.setIcon(new
        ImageIcon(scaledImage));

        JLabel nameLabel = new
        JLabel(soldierNames[i], SwingConstants.CENTER);
        nameLabel.setFont(new Font("맑은 고딕",
        Font.BOLD, 16));

        JLabel statsLabel = new JLabel("만족도: "
        + satisfactionMap.get(soldierNames[i]) + " | 훈련 점수:
        " + taskPointsMap.get(soldierNames[i]) + " | 휴가: " +

```

```

leaveMap.get(soldierNames[i]), SwingConstants.CENTE
R);
        statsLabel.setFont(new Font("맑은 고딕",
Font.PLAIN, 14));

        JButton actionButton = new
JButton(soldierNames[i] + " 훈련");
        int index = i;
        actionButton.addActionListener(new
ActionListener() {
            @Override
            public void
actionPerformed(ActionEvent e) {
                if
(onLeaveSoldiers.contains(soldierNames[index]))
return; // 휴가 중인 병사는 무시

                int satisfaction =
satisfactionMap.get(soldierNames[index]);
                int points =
taskPointsMap.get(soldierNames[index]);
                satisfaction -= 3;
                points += 5;

satisfactionMap.put(soldierNames[index], satisfaction);

taskPointsMap.put(soldierNames[index], points);
                statsLabel.setText("만족도: " +
satisfaction + " | 훈련 점수: " + points + " | 휴가: " +
leaveMap.get(soldierNames[index]));
            }
        });

        individualPanel.add(imageLabel,
BorderLayout.CENTER);
        individualPanel.add(nameLabel,
BorderLayout.NORTH);
        individualPanel.add(statsLabel,
BorderLayout.SOUTH);
        individualPanel.add(actionButton,
BorderLayout.EAST);

```

```

        soldierPanel.add(individualPanel);
    }

    gamePanel.add(soldierPanel,
BorderLayout.CENTER);

    JButton nextDayButton = new JButton("다음 날
진행");
    JButton backToMenuButton = new
JButton("메인 메뉴로");
    JPanel buttonPanel = new JPanel(new
GridLayout(1, 2, 10, 10));
    buttonPanel.add(nextDayButton);
    buttonPanel.add(backToMenuButton);
    addMuteButton(buttonPanel);
    gamePanel.add(buttonPanel,
BorderLayout.SOUTH);

    frame.getContentPane().removeAll();
    frame.add(gamePanel);
    frame.revalidate();
    frame.repaint();

    final int[] day = {1};
    nextDayButton.addActionListener(new
ActionListener() {
        @Override
        public void actionPerformed(ActionEvent
e) {

            day[0]++;
            dayLabel.setText("Day: " + day[0]);

            // soldierPanel 업데이트와 GUI
업데이트를 같은 스레드에서 처리
            updateSoldierStates(soldierPanel);
            updateAllGUIStates();

            if (day[0] % 7 == 0) {
                triggerRandomEvent(frame);
            }
        }
    });

```

```

        if (day[0] > 30) {
            showGameResults(frame);
        }
    });

```

```

        backToMenuButton.addActionListener(new
        ActionListener() {
            @Override
            public void actionPerformed(ActionEvent
            e) {
                JOptionPane.showMessageDialog(frame, "메인 메뉴로
                돌아갑니다. 관리할 메뉴를 선택하세요!", "메인 메뉴",
                JOptionPane.INFORMATION_MESSAGE);
                createShowGUI(frame);
            }
        });
    }

```

```

        private static synchronized void
        updateAllGUIStates() {
            System.out.println("GUI 상태가
            업데이트되었습니다. 만족도 및 휴가 정보가
            동기화됩니다.");
        }
    }

```

4. 실행 결과 화면

5. 느낀 점

처음에는 그냥 휴가 마일리지 관리로 코드를 작성하였지만, 군인에게 중요한 휴가로 게임을 만들고자 육군 말년 병장의 훈련과 휴가를 적용하여 이 시뮬레이터에 표현하였습니다. 병사들의 상태를 관리하기 위해 HashMap과 같은 자료구조를 활용해 데이터를 효율적으로 저장하고 업데이트하는 것이 중요하다는 것을 느꼈습니다.

병사 상태가 복잡하게 얽혀 있어 초기 설계 단계에서 각 상태 간의 상호작용을 명확히 정의하는 것이 매우 중요하다는 것을 알게 되었습니다.

병사들의 상태를 사용자가 직관적으로 파악할 수 있도록 GUI를 설계하는 과정이 어려우면서도 흥미로웠습니다. 특히, 병사들의 이미지를 사용하여 상태를 시각적으로 나타내고, 각 병사의 만족도, 훈련 점수, 휴가 상태를 GUI에 표시함으로써 경험을 쌓았습니다.

병사들의 상태가 변할 때마다 동기화하여 사용자가 즉각적인 피드백을 받을 수 있도록 하는 것이 중요했습니다 (updateSoldierStates).

단순한 관리 시뮬레이션에 그치지 않고, 랜덤 이벤트와 같은 변수를 추가함으로써 게임의 재미 요소를 높이는 것이 중요하다는 것을 느꼈습니다. 랜덤 이벤트(triggerRandomEvent)는 병사들의 상태를 변화시켜 사용자가 전략적으로 결정을 내리도록 만들었으며, 이 과정에서 게임 설계의 재미를 느꼈습니다.

기상 나팔을 배경 음악으로 추가하여 몰입감을 높이려는 시도는 색다른 경험이었습니다. 사용자가 병사 상태를 조작할 수 있는 버튼과 상태 변화 결과를 확인할 수 있는 창을 구현하며, 사용자 선택이 게임 결과에 직결된다는 것이 구현하였습니다.

동기화 작업이 이 코드를 짜면서 매우 어려웠습니다. 예를 들어 휴가중인데도, 만족도가 올라가거나 훈련점수가 올라가는 상황, 일정 진행중에 만족도 관리를 누르면 만족도가 게임과 다른 상황 등등 다른 시스템처럼 따로따로 구현되어서 구현하는 것이 매우 힘들었습니다. 상황에 맞게 같이 만족도가 같이 진행되는 시스템을 구현하였습니다. 아직도 코드를 짜면서 동기화 하는 것을 잘 모르겠습니다. 동기화하는 작업들을 많이해보면 좋았을 것 같다고 생각합니다.

그리고 만족도가 가장 높은 병사에게 휴가 1일을 줄려고 하였지만 많은 제약이 생겨서 하지 못하였습니다. 휴가가 나간 병사가만족도가 가장 높으면 휴가 1일을 줘야하는데 이미 휴가 예정이므로 다른 못나가는 이들에게 주는 시스템을 만들려고 하였는데 생각이 잘안나서 구현을 못하였습니다.

메뉴 버튼 이런 다양한 시스템들을 구현하면서 다음에도 이런 시스템을 작성하라고 하면 경험을 쌓은 상태로 할수 있어서 좋은 기회라고 생각합니다. 이런 과제를 내주신 권영미 교수님에게 감사합니다. 하면서 옛 병장 시절 생각도 나서 자바로도 추억을 생각하면서 게임을 만들 수 있었습니다.



충남대학교

REPORT 03

<군대 휴가 관리>

이름/학번 : 오석준/202002810

날짜 : 2024. 11. 23

이메일 : lose8468@naver.com

핸드폰번호 : 01073028468

1. 문제 정의

군대에서는 병사들의 휴가 관리가 매우 중요한 업무입니다. 저는 마이리지를 활용한 휴가 관리로 대체 하였습니다. 휴가 일수를 효율적으로 관리하고, 병사들이 얼마나 많은 휴가를 사용했는지 또는 추가적인 보너스 휴가가 필요한지 등을 파악하는 시스템을 마련하기 위해 코드를 작성하였습니다.

2. 해결 방법

먼저 프로그램을 사용하기 위해 로그인 기능을 구현하여, 비밀번호 인증 절차를 거치게 함으로써 보안을 강화했습니다.

그래픽 사용자 인터페이스(GUI)를 제공합니다. GUI를 통해 로그인을 진행하고, 휴가 조정, 병사 정보 조회 등을 손쉽게 수행할 수 있습니다.

병사들의 이름, 계급, 병과, 신체 정보, 휴가 관리, 징계 관리 등을 포함한 인적사항을 관리하는 정보를 저장하였습니다. HashMap과 같은 자료구조를 사용해 각 병사 정보를 저장하고 접근합니다.

휴가 데이터는 파일에서 읽어와 저장됩니다.

휴가 조정 시, 병사의 계급에 따라 추가 보너스 휴가가 자동으로 계산되어 반영됩니다.

병사 정보를 탐색하기 위해 콤보박스를 통해 병사를 선택하거나, 좌우 화살표 버튼을 이용해 병사 목록을 순차적으로 탐색할 수 있는 기능을 구현했습니다. 병사 사진, 인적사항, 휴가 기록, 징계 및 표창 내역 등을 한 눈에 보게 하였습니다.

3. 소스 코드

```
import java.awt.*;

import java.awt.event.*;

import javax.swing.*;

import java.io.*;

import java.util.*;

public class Military_leave_system {
```

```
private static HashMap<String, Vector<Integer>> map = new HashMap<String, Vector<Integer>>();
```

```
private static HashMap<String, String> rankMap = new HashMap<String, String>();
```

```
private static int currentIndex = 0;
```

```
private static String[] soldierNames = {"최종훈", "김태우", "군건이", "나백중"};
```

```
private static String[] soldierRanks = {"병장", "상병", "일병", "이병"};
```

```
private static String[] soldierImages = {"./images/최종훈.png", "./images/김태우.png", "./images/군건이.png", "./images/나백중.png"};
```

```
private static String[] soldierSpecialty = {"보병", "공병", "포병", "보급병"};
```

```
private static String[] soldierPhysical = {"신장: 175cm, 체중: 70kg", "신장: 190cm, 체중: 89kg", "신장: 175cm, 체중: 72kg", "신장: 174cm, 체중: 70kg"};
```

```
private static String[] soldierDisciplinary = {"상점 3, 벌점 1", "상점 1, 벌점 0", "상점 2, 벌점 2", "상점 0, 벌점 1"};
```

```
public static void main(String[] args) {
```

```
    rankMap.put("최종훈", "병장");
```

```
    rankMap.put("김태우", "상병");
```

```
    rankMap.put("군건이", "일병");
```

```
    rankMap.put("나백중", "이병");
```

```
    createShowGUI();
```

```
}
```

```
private static void createShowGUI() {
```

```
    JFrame frame = new JFrame("군대 휴가 관리 프로그램");
```

```
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```

frame.setSize(800, 600);

JPanel panel = new JPanel(new GridLayout(3, 1, 10, 10));
JLabel passwordLabel = new JLabel("비밀번호를 입력하세요: ");
JPasswordField passwordField = new JPasswordField(10);
JButton loginButton = new JButton("로그인");

panel.add(passwordLabel);
panel.add(passwordField);
panel.add(loginButton);

frame.add(panel);
frame.setVisible(true);

loginButton.addActionListener(e -> {
    String correctPassword = "1q2w3e4r1!";
    String enteredPassword = new String(passwordField.getPassword());

    if (!enteredPassword.equals(correctPassword)) {
        JOptionPane.showMessageDialog(frame, "비밀번호가 틀렸습니다. 프로그램을 종료합니다.", "오류", JOptionPane.ERROR_MESSAGE);
        System.exit(0);
    } else {
        frame.getContentPane().removeAll();
        showMenu(frame);
    }
});

```

```
}
```

```
private static void showMenu(JFrame frame) {  
    JPanel panel = new JPanel(new GridLayout(5, 1, 10, 10));  
  
    JButton readButton = new JButton("1. 휴가 파일 읽기");  
    JButton adjustButton = new JButton("2. 휴가 조정");  
    JButton statusButton = new JButton("3. 휴가 현황 출력");  
    JButton comboBoxButton = new JButton("4. 병사 인적사항");  
    JButton exitButton = new JButton("5. 종료");  
  
    panel.add(readButton);  
    panel.add(adjustButton);  
    panel.add(statusButton);  
    panel.add(comboBoxButton);  
    panel.add(exitButton);  
  
    frame.getContentPane().removeAll();  
    frame.add(panel);  
    frame.revalidate();  
    frame.repaint();  
  
    readButton.addActionListener(e -> readLeaveFile());  
    adjustButton.addActionListener(e -> adjustLeaveWithBonus());  
    statusButton.addActionListener(e -> printLeaveStatus());  
    comboBoxButton.addActionListener(e -> showSoldierComboBox(frame));  
    exitButton.addActionListener(e -> System.exit(0));  
}
```

```
}
```

```
private static void showSoldierComboBox(JFrame frame) {  
    JPanel panel = new JPanel(new BorderLayout(10, 10));  
  
    JLabel comboBoxLabel = new JLabel("병사를 선택해주세요: ",  
SwingConstants.CENTER);  
  
    JComboBox<String> soldierComboBox = new JComboBox<>(soldierNames);  
    soldierComboBox.setPreferredSize(new Dimension(150, 20));  
  
    JButton selectButton = new JButton("선택");  
  
    panel.add(comboBoxLabel, BorderLayout.NORTH);  
    panel.add(soldierComboBox, BorderLayout.CENTER);  
    panel.add(selectButton, BorderLayout.SOUTH);  
  
    frame.getContentPane().removeAll();  
    frame.add(panel);  
    frame.revalidate();  
    frame.repaint();  
  
    selectButton.addActionListener(e -> {  
        int selectedIndex = soldierComboBox.getSelectedIndex();  
        if (selectedIndex != -1) {  
            currentIndex = selectedIndex;  
            showSoldierDetails(frame);  
        } else {  
            JOptionPane.showMessageDialog(frame, "군인을 선택하세요.", "오류",
```

```
JOptionPane.ERROR_MESSAGE);
```

```
    }
```

```
});
```

```
}
```

```
private static void showSoldierDetails(JFrame frame) {
```

```
    JPanel mainPanel = new JPanel(new BorderLayout(10, 10));
```

```
    mainPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
```

```
    JPanel topPanel = createTopPanel();
```

```
    JPanel bottomPanel = createBottomPanel();
```

```
    mainPanel.add(topPanel, BorderLayout.CENTER);
```

```
    mainPanel.add(bottomPanel, BorderLayout.SOUTH);
```

```
    JPanel buttonPanel = new JPanel(new FlowLayout());
```

```
    JButton leftButton = new JButton("<");
```

```
    JButton rightButton = new JButton(">");
```

```
    JButton backButton = new JButton("뒤로 가기");
```

```
    buttonPanel.add(leftButton);
```

```
    buttonPanel.add(rightButton);
```

```
    buttonPanel.add(backButton);
```

```
    frame.getContentPane().removeAll();
```

```
    frame.setLayout(new BorderLayout());
```

```
    frame.add(mainPanel, BorderLayout.CENTER);
```

```
    frame.add(buttonPanel, BorderLayout.SOUTH);
```

```

frame.revalidate();

frame.repaint();

leftButton.addActionListener(e -> {
    currentIndex = (currentIndex - 1 + soldierNames.length) %
soldierNames.length;

    showSoldierDetails(frame);
});

rightButton.addActionListener(e -> {
    currentIndex = (currentIndex + 1) % soldierNames.length;

    showSoldierDetails(frame);
});

backButton.addActionListener(e -> showMenu(frame));

frame.addKeyListener(new KeyAdapter() {
    @Override
    public void keyPressed(KeyEvent e) {
        if (e.getKeyCode() == KeyEvent.VK_LEFT) {
            currentIndex = (currentIndex - 1 + soldierNames.length) %
soldierNames.length;

            showSoldierDetails(frame);
        } else if (e.getKeyCode() == KeyEvent.VK_RIGHT) {
            currentIndex = (currentIndex + 1) % soldierNames.length;

            showSoldierDetails(frame);
        }
    }
}

```

```

});

frame.setFocusable(true);

frame.requestFocusInWindow();

}

private static JPanel createTopPanel() {

    JPanel photoPanel = new JPanel();

    photoPanel.setBorder(BorderFactory.createTitledBorder("사진"));

    JLabel soldierImageLabel = new JLabel(new ImageIcon(new
    ImageIcon(soldierImages[currentIndex]).getImage().getScaledInstance(250, 250,
    Image.SCALE_SMOOTH)));

    photoPanel.add(soldierImageLabel);

    JPanel personalInfoPanel = new JPanel(new GridLayout(5, 1));

    personalInfoPanel.setBorder(BorderFactory.createTitledBorder("인적사항"));

    personalInfoPanel.add(new JLabel("이름: " + soldierNames[currentIndex]));

    personalInfoPanel.add(new JLabel("계급: " + soldierRanks[currentIndex]));

    personalInfoPanel.add(new JLabel("병과: " + soldierSpecialty[currentIndex]));

    personalInfoPanel.add(new JLabel("신체: " + soldierPhysical[currentIndex]));

    personalInfoPanel.add(new JLabel("연락처: 없음"));

    JPanel topPanel = new JPanel(new GridLayout(1, 2, 10, 10));

    topPanel.add(photoPanel);

    topPanel.add(personalInfoPanel);

    return topPanel;

}

```

```

private static JPanel createBottomPanel() {

    JPanel leavePanel = new JPanel();

    leavePanel.setBorder(BorderFactory.createTitledBorder("휴가 기록"));

    JTextArea leaveTextArea = new JTextArea(5, 20);

    Vector<Integer> leaveVector = map.get(soldierNames[currentIndex]);

    leaveTextArea.setText((leaveVector != null && !leaveVector.isEmpty()) ?
leaveVector.stream().map(String::valueOf).reduce((a, b) -> a + "일, " + b + "일
").orElse("") : "휴가 기록이 없습니다.");

    leaveTextArea.setEditable(false);

    leavePanel.add(new JScrollPane(leaveTextArea));

    JPanel recordsPanel = new JPanel(new GridLayout(2, 1));

    recordsPanel.setBorder(BorderFactory.createTitledBorder("징계 및 표창 기록"));

    recordsPanel.add(new JLabel("징계 기록: " +
soldierDisciplinary[currentIndex]));

    JPanel bottomPanel = new JPanel(new GridLayout(1, 2, 10, 10));

    bottomPanel.add(leavePanel);

    bottomPanel.add(recordsPanel);

    return bottomPanel;
}

```

```

private static void readLeaveFile() {

```

```

    try (FileInputStream fin = new FileInputStream("./leaveinput.txt");

        InputStreamReader in = new InputStreamReader(fin, "UTF-8");

```

```

Scanner sc = new Scanner(in) {

    map.put("최종훈", new Vector<>());
    map.put("김태우", new Vector<>());
    map.put("굳건이", new Vector<>());
    map.put("나백중", new Vector<>());

    while (sc.hasNextLine()) {

        String[] tokens = sc.nextLine().split(" ");

        String name = tokens[0];

        Vector<Integer> v = map.get(name);

        if (v == null) {

            JOptionPane.showMessageDialog(null, name + "은 없는 군인입니
다.", "오류", JOptionPane.ERROR_MESSAGE);

            continue;

        }

        for (int i = 1; i < tokens.length; i++) {

            v.add(Integer.parseInt(tokens[i]));

        }

    }

    JOptionPane.showMessageDialog(null, "휴가 파일을 성공적으로 읽었습니다.",
"성공", JOptionPane.INFORMATION_MESSAGE);

    } catch (IOException e) {

        JOptionPane.showMessageDialog(null, "파일 읽기 오류 : " + e.getMessage(),
"오류", JOptionPane.ERROR_MESSAGE);

    }
}

```

```
}
```

```
private static void adjustLeaveWithBonus() {
```

```
    try (FileInputStream fin = new FileInputStream("./leaveadjust.txt");
```

```
        InputStreamReader in = new InputStreamReader(fin, "UTF-8");
```

```
        Scanner sc = new Scanner(in)) {
```

```
        while (sc.hasNextLine()) {
```

```
            String[] tokens = sc.nextLine().split(" ");
```

```
            String name = tokens[0];
```

```
            int leaveAdjustment = Integer.parseInt(tokens[1]);
```

```
            Vector<Integer> v = map.get(name);
```

```
            if (v == null) {
```

```
                JOptionPane.showMessageDialog(null, name + "은 없는 군인입니  
다.", "오류", JOptionPane.ERROR_MESSAGE);
```

```
                continue;
```

```
            }
```

```
            // 계급 보너스
```

```
            String rank = rankMap.get(name);
```

```
            int bonus = getLeaveBonusByRank(rank);
```

```
            v.add(leaveAdjustment + bonus);
```

```
            JOptionPane.showMessageDialog(null, name + "의 휴가가 조정되었습니  
다. (보너스 포함: " + bonus + "일)", "성공", JOptionPane.INFORMATION_MESSAGE);
```

```

    }

    } catch (IOException e) {

        JOptionPane.showMessageDialog(null, "파일 읽기 오류 : " + e.getMessage(),
"오류", JOptionPane.ERROR_MESSAGE);

    }

}

```

```

private static int getLeaveBonusByRank(String rank) {

    switch (rank) {

        case "병장": return 5;

        case "상병": return 3;

        case "일병": return 2;

        case "이병": return 1;

        default: return 0;

    }

}

```

```

private static void printLeaveStatus() {

    StringBuilder leaveSummary = new StringBuilder();

    String topSoldier = "";

    int maxLeave = 0;

    for (String name : map.keySet()) {

        int totalLeave =
map.get(name).stream().mapToInt(Integer::intValue).sum();

        leaveSummary.append("(").append(name).append(":").append(totalLeave).append(" ");

        if (totalLeave > maxLeave) {

```

```

        maxLeave = totalLeave;

        topSoldier = name;
    }
}

JOptionPane.showMessageDialog(null, leaveSummary.toString(), "휴가 현황",
JOptionPane.INFORMATION_MESSAGE);

JOptionPane.showMessageDialog(null, "가장 많은 휴가를 가진 군인은 " +
topSoldier + "입니다.", "휴가 현황", JOptionPane.INFORMATION_MESSAGE);
}
}

```

4. 실행 결과 화면

5. 느낀 점

이 프로그램을 통해 군대 내 병사들의 휴가 관리가 더욱 쉽고 편리하게 관리할 수 있습니다. Java의 GUI와 파일 입출력 기능을 적극적으로 활용는 프로그램을 개발할 수 있었습니다. ActionListener와 ItemListener 등을 활용한 이벤트 처리 방식을 통해 이벤트 기반 프로그래밍에 대해 이해할 수 있는 좋은 기회가 되었습니다. 컴포넌트를 통해 사용자가 쉽게 사용하는 프로그램을 구현하면서 이

벤트 리스너의 역할을 알 수 있었고, 이를 통해 어떻게 사용해야 하는지를 알 수 있었습니다. 이렇게 프로그램이 사용된다면 일일이 작성할 필요없이 GUI로 보기에도 좋고 한번에 파일이 입력되는 프로그램으로 매우 편리할 것 같습니다. 매우 어려웠지만 이것을 작성하면서 매우 뿌듯했습니다. 시간이 더 많이 있었다면 더 다른 방법들도 고민하고 사용했을 것 같습니다.

REPORT 02

<txt파일 불러오기>



충남대학교

이
/
번

오석준/202002810

름
학
:

날짜 : 2024. 11. 16

이메일 : lose8468@naver.com

핸드폰번호 : 01073028468

1. 문제 정의

고객의 마일리지를 관리하는 시스템에서, 직접 입력하는 것이 아닌 마일리지 정보를 텍스트 파일에서 읽어서 처리하는 프로그램이 필요한 과제입니다. 프로그램은 고객의 이름과 마일리지 정보를 포함한 텍스트 파일(mileageinput.txt)을 읽어와 각 고객의 초기 마일리지 정보를 저장하고, 별도의 파일(mileagededuct.txt)을 통해 특정 고객의 마일리지를 차감해야 합니다.

2. 해결 방법

문제를 해결하기 위해 Java 언어를 사용하여 두 개의 텍스트 파일(mileageinput.txt, mileagededuct.txt)로부터 데이터를 읽어들이는 프로그램을 구현했습니다. 이 프로그램은 다음과 같은 과정을 통해 문제를 해결하였습니다.

-마일리지 입력 파일 읽기:

mileageinput.txt 파일을 읽어 각 고객의 이름과 마일리지 값을 불러옵니다. 이를 위해 FileInputStream과 InputStreamReader를 사용하였고, 파일 내의 데이터를 읽어오기 위해 Scanner 객체를 활용하였습니다. 파일에는 고객 이름과 그에 해당하는 마일리지 값이 줄 단위로 기록되어 있으며, 프로그램은 이를 통해 마일리지 상태를 설정합니다.

-마일리지 정보 저장:

각 고객의 마일리지 정보를 저장하기 위해 HashMap과 Vector를 사용하였습니다. HashMap은 고객의 이름을 키로, 각 고객의 마일리지 내역을 값으로 저장하며, Vector는 마일리지를 리스트 형태로 누적 관리할 수 있도록 합니다. 이를 통해 마일리지 정보를 효율적으로 관리하고, 고객별로 누적된 마일리지 값을 손쉽게 접근할 수 있도록 했습니다.

-마일리지 차감 파일 읽기:

mileagededuct.txt 파일을 사용해 특정 고객의 마일리지를 차감하

는 작업을 수행합니다. 이 파일 역시 FileInputStream과 InputStreamReader, Scanner를 이용해 데이터를 읽어오며, 파일에는 차감할 고객의 이름과 차감할 마일리지 값이 기록되어 있습니다. 프로그램은 해당 데이터를 기반으로 고객의 총 마일리지에서 차감 작업을 수행합니다. 만약 차감할 마일리지의 현재 보유한 마일리지보다 많을 경우, 프로그램은 경고 메시지를 출력하고 차감 작업을 중단하는 것으로 구현하였습니다.

각 파일을 모두 읽고 난 후에는 Scanner와 InputStreamReader 객체를 close() 메소드를 이용해 닫았습니다.

-마일리지 현황 출력:

모든 마일리지 누적 및 차감 작업이 완료된 후, 각 고객의 총 마일리지 현황을 출력합니다. 이와 함께 가장 많은 마일리지를 보유한 고객을 표시하여 마일리지 관리의 데이터들을 볼 수 있도록 구현하였습니다.

「첨부 파일」

1. 
mileageinput.txt

2. 
mileagededuct.txt

3. 소스 코드

```
import java.io.*;
```

```

import java.util.*;

public class MileageProgram {

    public static void main(String[] args) {
        InputStreamReader in = null;
        FileInputStream fin = null;
        Scanner sc = null;
        try {

            fin = new
FileInputStream("D:\\java\\mileageinput.txt");
            in = new InputStreamReader(fin, "UTF-8");
            sc = new Scanner(in);

            HashMap<String, Vector<Integer>> map = new
HashMap<String, Vector<Integer>>();

            map.put("윤단비", new Vector<Integer>());
            map.put("김은비", new Vector<Integer>());
            map.put("전아린", new Vector<Integer>());
            map.put("하여린", new Vector<Integer>());

            System.out.println("*** 마일리지 관리 프로그램입니다.
***");
            while (sc.hasNextLine()) {
                String line = sc.nextLine();
                if (line.equals("그만")) {
                    break;
                }
                String[] tokens = line.split(" ");
                String name = tokens[0];

                Vector<Integer> v = map.get(name);

                if (v == null) {
                    System.out.println(name + "은 없는 학생입니
다.");
                    continue;
                }
            }
        }
    }
}

```

```

        for (int i = 0; i < tokens.length - 1; i++) {
            v.add(Integer.parseInt(tokens[i + 1]));
        }
    }

    fin = new
    FileInputStream("D:\\java\\mileagededuct.txt");
    in = new InputStreamReader(fin, "UTF-8");
    sc = new Scanner(in);

    while (sc.hasNextLine()) {
        System.out.print("차감할 이름과 마일리지>>");
        String line = sc.nextLine();
        String[] tokens = line.split(" ");
        String name = tokens[0];
        int mileageDeduction =
        Integer.parseInt(tokens[1]);

        Vector<Integer> v = map.get(name);

        if (v == null) {
            System.out.println(name + "은 없는 학생입니
다.");
            continue;
        }

        int currentTotalMileage = 0;
        for (int mileage : v) {
            currentTotalMileage += mileage;
        }

        if (currentTotalMileage < mileageDeduction) {
            System.out.println("마일리지가 부족합니다.");
        } else {
            for (int i = 0; i < v.size(); i++) {
                int mileage = v.get(i);
                if (mileage <= mileageDeduction) {
                    mileageDeduction -= mileage;
                    v.set(i, 0);
                } else {

```

```

                v.set(i, mileage - mileageDeduction);
                break;
            }
        }
        System.out.println(name + "의 마일리지가 차감
되었습니다.");
    }
}

sc.close();
in.close();
fin.close();

System.out.println("\n*** 마일리지 현황 ***");
Set<String> keys = map.keySet();
Iterator<String> it = keys.iterator();

String topStudent = "";
int maxMileage = 0;

String mileageSummary = "";

while (it.hasNext()) {
    String name = it.next();
    Vector<Integer> v = map.get(name);

    int totalMileage = 0;
    for (int mileage : v) {
        totalMileage += mileage;
    }

    mileageSummary += "(" + name + ":" +
totalMileage + ") ";

    if (totalMileage > maxMileage) {
        maxMileage = totalMileage;
        topStudent = name;
    }
}

System.out.println(mileageSummary);
System.out.println("가장 마일리지가 높은 고객은 " +

```

```
topStudent + "입니다.");  
        System.out.println("프로그램을 종료합니다.");  
    } catch (IOException e) {  
        System.out.println("파일 복사 오류 : " + e.getMessage()  
);  
    }  
}  
}
```

4. 실행 결과 화면

*** 마일리지 관리 프로그램입니다. ***

차감할 이름과 마일리지>>윤단비의 마일리지가 차감되었습니다.

차감할 이름과 마일리지>>하여린의 마일리지가 차감되었습니다.

차감할 이름과 마일리지>>김은비의 마일리지가 차감되었습니다.

차감할 이름과 마일리지>>전아린의 마일리지가 차감되었습니다.

*** 마일리지 현황 ***

(윤단비:700) (김은비:2000) (하여린:2600) (전아린:2500)

가장 마일리지가 높은 고객은 하여린입니다.

프로그램을 종료합니다.

5. 느낀 점

이번 프로그램을 구현하면서 파일 입출력을 통한 데이터 관리의 중요성을 깊이 느꼈습니다. 파일에서 데이터를 읽어와 프로그램 내에서 처리하는 방식은 대량의 데이터를 손쉽게 다룰 수 있는 장점이 있다는 것을 알 수 있었습니다. 또한, 직접 입력하지 않아도 되는 번거로움이 사라져서 매우 편했습니다. 그러나, 파일 형식이 조금이라도 일치하지 않으면 프로그램이 데이터를 제대로 읽어오지 못해 오류가 발생할 수 있습니다. 따라서 이러한 데이터를 검증하는 절차도 필요함을 알게 되었습니다.

또한, 자바에서 제공하는 다양한 파일 입출력 관련 클래스(FileInputStream, InputStreamReader, Scanner)를 사용하면서 입출력 값 관리를 효율적으로 하는 방법을 배웠습니다. 파일을 다루는 코드에서 close() 메소드를 사용해야 하는 것도 깨달았습니다. 마일리지 차감 로직을 구현하면서도 여러 가지 고려할 점이 있었습니다. 고객의 마일리지가 부족할 경우 이를 어떻게 처리할지, 어떤 메시지를 사용자에게 제공할지, 이러한 상황들에 대해 고민해야 했습니다. 이러한 예외 상황을 처리하는 것이 프로그램 작성하는 것에서 매우 중요하다는 것을 배웠습니다. 또한, 고객별로 여러 개의 마일리지 내역을 관리하기 위해 Vector를 사용하였는데, 이를 통해 마일리지의 추가와 차감이 쉽게 이루어질 수 있었습니다. 파일 입출력 과정은 이름과 마일리지를 수동으로 입력할 필요 없이 자동으로 파일을 읽어 처리함으로써 사용자가 편리하게 출력 할 수 있습니다. 파일 입출력을 통해 데이터를 관리하는 방식이 다양한 실제 상황을 응용한 프로그램에서 어떻게 활용될 수 있는지에 대한 이해도를 높일 수 있었습니다.

REPORT 01

<HashMap 응용>



충남대학교

이름/학번 : 오석준/202002810

날짜 : 2024. 11. 11

이메일 : lose8468@naver.com

핸드폰번호 : 01073028468

1. 문제 정의

고객들은 마일리지를 누적하며, 누적된 마일리지를 사용하여 물품 구매나 특정 서비스 혜택을 받을 수 있습니다. 따라서 고객의 마일리지를 누적하고 차감할 수 있는 시스템이 필요합니다. 이 시스템은 다음과 같은 기능을 제공해야 합니다:

1. 고객의 마일리지 누적 및 차감
2. 고객 개인의 총 마일리지 조회
3. 가장 많은 마일리지를 보유한 고객 확인

2. 해결 방법

본 프로그램은 HashMap 자료구조를 사용하여 고객의 마일리지 정보를 관리합니다. 프로그램의 주요 해결 방법:

1.고객 마일리지 누적: 사용자가 고객 이름과 누적할 마일리지를 입력하면 해당 고객의 마일리지가 추가됩니다. 마일리지 정보는 고객 이름을 키로 하고, 적립된 각 마일리지를 값으로 가지는 Vector 형태로 저장됩니다.

2.마일리지 차감: 고객의 마일리지를 사용할 수 있게 사용자가 차감할 고객 이름과 마일리지를 입력하면 해당 고객의 마일리지에서 차감됩니다. 차감 과정에서 고객의 총 마일리지가 부족하면, 부족하다는 메시지를 출력하고 차감하지 않습니다.

3.마일리지 현황 출력: 프로그램은 모든 고객의 마일리지 현황을 출력합니다. 가장 많은 마일리지를 보유한 고객을 별도로 표시합니다.

3. 소스 코드

```
import java.util.*;

public class MileageProgram {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        HashMap<String, Vector<Integer>> map = new
        HashMap<String, Vector<Integer>>();

        map.put("윤단비", new Vector<Integer>());
        map.put("김은비", new Vector<Integer>());
        map.put("전아린", new Vector<Integer>());
        map.put("하여린", new Vector<Integer>());

        System.out.println("*** 마일리지 관리 프로그램입니다.
***");

        while (true) {

            System.out.print("이름과 마일리지>>");

            String line = scanner.nextLine();

            if (line.equals("그만")) {

                break;

            }

            String[] tokens = line.split(" ");
```

```

String name = tokens[0];

Vector<Integer> v = map.get(name);

if (v == null) {
    System.out.println(name + "은 없는 학생입니
다.");
    continue;
}

for (int i = 0; i < tokens.length - 1; i++) {
    v.add(Integer.parseInt(tokens[i + 1]));
}
}

while (true) {
    System.out.print("차감할 이름과 마일리지>>");

    String line = scanner.nextLine();

    if (line.equals("그만")) {
        break;
    }
}

```

```

    }

    String[] tokens = line.split(" ");

    String name = tokens[0];

    int mileageDeduction = Integer.parseInt(tokens[1]);

    Vector<Integer> v = map.get(name);

    if (v == null) {
        System.out.println(name + "은 없는 학생입니
다.");

        continue;
    }

    int currentTotalMileage = 0;

    for (int mileage : v) {
        currentTotalMileage += mileage;
    }

    if (currentTotalMileage < mileageDeduction) {
        System.out.println("마일리지가 부족합니다.");
    } else {

```

```

        for (int i = 0; i < v.size(); i++) {
            int mileage = v.get(i);
            if (mileage <= mileageDeduction) {
                mileageDeduction -= mileage;
                v.set(i, 0);
            } else {
                v.set(i, mileage - mileageDeduction);//
마일리지 차감

                break;
            }
        }

        System.out.println(name + "의 마일리지가 차감되
었습니다.");
    }
}

```

```

System.out.println("\n*** 마일리지 현황 ***");

```

```

Set<String> keys = map.keySet();

```

```

Iterator<String> it = keys.iterator();

```

```
String topStudent = "";

int maxMileage = 0;

String mileageSummary = "";

while (it.hasNext()) {

    String name = it.next();

    Vector<Integer> v = map.get(name);

    int totalMileage = 0;

    for (int mileage : v) {

        totalMileage += mileage;// 마일리지 누적

    }

    mileageSummary += "(" + name + ":" + totalMileage
+ ") ";

    if (totalMileage > maxMileage) {

        maxMileage = totalMileage;

        topStudent = name;

    }

}
```

```
}
```

```
}
```

```
System.out.println(mileageSummary);
```

```
System.out.println("가장 마일리지가 높은 고객은 " +  
topStudent + "입니다.");
```

```
System.out.println("프로그램을 종료합니다.");
```

```
}
```

```
}
```

4. 실행 결과 화면

*** 마일리지 관리 프로그램입니다. ***

이름과 마일리지>>전아린 2000

이름과 마일리지>>하여린 1800

이름과 마일리지>>김은비 2500

이름과 마일리지>>전아린 1500

이름과 마일리지>>윤단비 2700

이름과 마일리지>>하여린 1800

이름과 마일리지>>그만

차감할 이름과 마일리지>>윤단비 2000

윤단비의 마일리지가 차감되었습니다.

차감할 이름과 마일리지>>그만

*** 마일리지 현황 ***

(윤단비:700) (김은비:2500) (하여린:3600) (전아린:3500)

가장 마일리지가 높은 고객은 하여린입니다.

프로그램을 종료합니다.

5. 느낀 점

마일리지 관리 프로그램을 작성하면서 여러 가지 중요한 점을 배우고 깨달았습니다. 데이터 구조의 적절한 선택이 프로그램의 효율성에 큰 영향을 미친다는 것입니다. 'HashMap'을 사용해 각 고객의 이름과 마일리지를 관리함으로써 빠르고 쉽게 데이터를 조회하고 수정할 수 있었습니다. 특히, 고객의 마일리지를 효율적으로 누적하고 관리하기 위해 'HashMap'과 'Vector'의 조합을 사용하여 데이터를 구조화하였고, 이를 통해 고객별로 적립된 마일리지 항목들을 관리할 수 있었습니다. 특히 고객의 마일리지를 차감하는 부분에서 여러 가지 상황을 고려해야 했습니다. 고객이 여러 번에 걸쳐 마일리지를 적립하는 경우, 이 마일리지를 차감할 때 총합이 차감하고자 하는 금액보다 충분한지 확인하고, 적절히 차감하는 코드를 구현하는 것이 어려웠습니다. 마일리지를 누적하는 것은 차감하는 것보다 쉽게 구현하였습니다. 마일리지를 차감하기 위한 코드를 작성하는 것이 생각보다 어려웠습니다.

Iterator를 인터페이스를 사용하여 컬렉션의 순차 검색할 때 매우 유용하였습니다. 모든 요소들을 출력하는 데 매우 간단하게 코드를 작성할 수 있는 것이 가독성이 좋습니다. 'HashMap'을 이용하여 키와 값이라는 요소를 이용하여 위치는 알 수 없지만 키를 이용하여 값을 리턴할 수 있는 것에서 매우 신기했습니다. 위치 이동이 필요 없어서 간단한 코드로 이루어지고, 키로만 검색해야 하는 것에서 'HashMap'에 대해서 빠르게 검색하고 삽입 삭제할 수 있는 컬렉션이라는 것을 깨달았습니다